Somewhat efficient blind recommender system for the MovieLens dataset (extended abstract)

Lorenzo Rovida
lorenzo.rovida@polito.it

Politecnico di Torino, Turin, Italy

1 Introduction

Recommender systems [Agg16] are designed to predict and surface content, products, or information that a user is likely to find relevant. However, this typically requires the service provider to have full access to user data in order to make personalized suggestions, which raises critical privacy concerns. In the recommender systems community, the MovieLens 100k dataset [HK15] plays a role similar to MNIST – a standard benchmark that most methods are evaluated on. Interestingly, this dataset has seen little use in cryptographic applications based on fully homomorphic encryption (FHE), which is a gap we aim to address.

Previous works. Some prior work [KKK+18,CBZ25] has tackled MovieLens under FHE, but with notable limitations. Both require the client to perform the final classification step, meaning the recommendation scores must be decrypted before a result can be returned.

This design choice comes with several problems: (i) returning the full list of scores can expose the system to attacks that attempt to extract the learned weights, (ii) it increases computational cost since a large number of values need to be sent back, and (iii) the top-k results cannot be used for any further server-side computation — for instance, to privately retrieve metadata about the recommended movies. The natural fix is to evaluate the argmax directly over the encrypted scores, but this is far from straightforward in FHE.

Indexing and sorting in FHE. Sorting has historically been one a challenge in FHE. In the last couple of years, however, new exciting and elegant solutions [MEHP25,KUYL25,RLB25] based on CKKS [CKKS17] have started to make sorting more feasible in practice. In this work, we take advantage of these recent developments to implement the argmax function directly within the FHE pipeline, with the goal of hopefully establishing a benchmark for future FHE-based recommender systems.

2 Methodology

Our setup is composed of two parties: the client and an honest-but-curious server. The proposal is composed of two different steps. The first refers to the training

process, which is performed by the server, and can be performed on clear data. In such case, on the MovieLens 100k dataset[1], which recommends among 1700 classes. The second one is performed under FHE, using the CKKS scheme and the *permutation-based* sorting approach á la [RLB25].

2.1 Learning the parameters

Our goal is to recommend the top-k (with k typically being 3 or 5) classes by finding the maximum indices of a scores vector $\boldsymbol{s} = W\boldsymbol{x} + \boldsymbol{b} \in \mathbb{R}^n$, with $\boldsymbol{x} \in \mathbb{R}^n$ being the user latent factor vector, $W \in \mathbb{R}^{n \times n}$ the item latent factor vector (or simply weight) and $\boldsymbol{b} \in \mathbb{R}^n$ some bias term. In MovieLens 100k, $n = 1700$. A simple SVD model allows us to derive some parameters W, b such that the RMSE is roughly 0.934. Since our main focus is on the FHE side, we treat these weights as given and do not further optimize the model.

2.2 Blindly predicting the top-k classes

Since we aim to use permutation-based sorting [RLB25], we first find some permutation $P \in \{0,1\}^{n \times n}$ such that the scores $\boldsymbol{s} \in \mathbb{R}^n$ are sorted as $P \cdot \boldsymbol{s}$, and then simply apply the permutation to a vector of values $(1, 2, \ldots, 1700)$ that correspond to the class labels of MovieLens 100k. The result can be obtained by masking the first k values.

An optimization for the top-k case. One issue with CKKS-based sorting is that speed and precision tend to decresase as the number of elements grows. Additionally, the permutation-based approach requires storing n^2 slots to sort a vector of size n. In our case, the scores vector has 1,700 entries, which would require up to 2,890,000 slots — clearly impractical.

To address this, we split the scores into 14 chunks of $n = 128$ values, each of which fits comfortably in a single ciphertext with $\Theta(n^2)$ slots In fact, this is the $n = 128$ case of [RLB25]. The sorting procedures are then run independently on each chunk — and can be parallelized. This yields 14 different permutation matrices, and each is applied to its corresponding class (sub)vector, resulting in 14 sorted vectors with their associated classes. Rather than reconstructing the full sorted vector, we extract only the top-k elements from each chunk, leaving us with $14k$ candidates, which are then fed into one final sorting step to produce the top-k elements of the original vector.

Lemma 1. *Let $\boldsymbol{b}_1, \boldsymbol{b}_2, \ldots, \boldsymbol{b}_n$ be a partition of a set $\boldsymbol{s}$ and let $k \in \mathbb{Z}^+$. Then the top-k elements of $\boldsymbol{s}$ are contained in the union of the top-k of all the $\boldsymbol{b}_i$.*

The proof can be found in Appendix A. Therefore, to obtain the global top-k of $\boldsymbol{s}$, it suffices to collect the top-k elements from each $\boldsymbol{b}_i$, yielding at most $n \cdot k$ candidates, and rank them globally.

[1] https://grouplens.org/datasets/movielens/100k/

References

Agg16. Charu C. Aggarwal. *Recommender systems*, volume 1. 2016.

CBZ25. Moontaha Nishat Chowdhury, Andre Bauer, and Minxuan Zhou. Efficient Privacy-Preserving Recommendation on Sparse Data using Fully Homomorphic Encryption . In *2025 IEEE International Conference on eScience (eScience)*, pages 1–9, 2025.

CKKS17. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology – ASIACRYPT 2017*, pages 409–437, 2017.

HK15. F. Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Trans. Interact. Intell. Syst.*, 5(4), 2015.

KKK$^+$18. Jinsu Kim, Dongyoung Koo, Yuna Kim, Hyunsoo Yoon, Junbum Shin, and Sungwook Kim. Efficient privacy-preserving matrix factorization for recommendation via fully homomorphic encryption. *ACM Transactions on Privacy and Security (TOPS)*, 21(4):1–30, 2018.

KUYL25. Seunghu Kim, Eymen Ünay, Ayse Yilmazer-Metin, and Hyung Tae Lee. Optimized rank sort for encrypted real numbers. Cryptology ePrint Archive, Paper 2025/1170, 2025.

MEHP25. Federico Mazzone, Maarten Everts, Florian Hahn, and Andreas Peter. *Efficient ranking, order statistics, and sorting under CKKS*. 2025.

RLB25. Lorenzo Rovida, Alberto Leporati, and Simone Basile. Lightweight sorting in approximate homomorphic encryption. Cryptology ePrint Archive, Paper 2025/1150, 2025.

A Proof of lemma 1

Proof. Suppose, by contradiction, that some element x belongs to the global top-k of $\boldsymbol{s}$ but is not in the top-k of its block $\boldsymbol{b}_i$. Then there exist at least k elements in $\boldsymbol{b}_i$ greater than x. Since $\boldsymbol{b}_i \subseteq \boldsymbol{s}$, these are also elements of $\boldsymbol{s}$ greater than x, so at least k elements of $\boldsymbol{s}$ are greater than x. This contradicts the assumption that x is among the k largest elements of $\boldsymbol{s}$. □